DATA ANALYTICS · PYTHON

Visual Exploration and Interactive Dashboards

Concept and practice guide

What you will learn in this guide

In this guide you will learn how to look at a new sales dataset from different angles and find what is going on in it, and then how to present your findings in an interactive dashboard and a short report. First, you will learn a fixed order for a first look at any data, so you know its size, its dates and its gaps before you draw anything. Second, you will learn which chart finds which pattern: a line chart for change over time, a bar chart for the days of the week, a heatmap for two categories at once. Third, you will learn to spot outliers, which are values far away from the rest, and to decide whether to keep or remove them, because deleting a real value is as harmful as keeping a wrong one. Fourth, you will build interactive Plotly charts that people can hover over, zoom into and filter, and combine four of them into one dashboard saved as a web page that anyone can open without Python. Next, you will learn Streamlit, a tool that turns the same charts into a small web app with filters, so a manager can choose an outlet or a month and watch every chart and number change. Finally, you will learn to write a one-page report that a manager with no technical background can act on. The advantage is that you can take any raw table, find the story in it and hand it over in a form people actually use, whether that is a saved page, a filterable app or a one-page report.

How to use this guide

1. Open one new Jupyter notebook. Run the practice-data cell from Appendix A once, at the top. It builds a table about a small café chain, which every example uses.
2. Read each section, then run its code in your own notebook. The text after each code block says what you should see.
3. Every function is explained in a table, argument by argument, so you know what each part of a line is for.
4. Do the “Your turn” box at the end of each section before you read on. The answers are in Appendix B; look at them only after you have tried.
5. Write what you find in your own words in a Markdown cell below each chart. A good answer has a claim, a number from the data and what it means.
6. When you finish, restart the kernel and run all cells from the top to check that nothing fails.

The practice data

The data is the daily sales record of a café chain with three outlets, for one year. Each row is the revenue of one item at one outlet on one day. A few unusual values are hidden in it on purpose, so you can practise finding them. The table lists the columns.

Column	Meaning
Date	The day of the sale, from 1 January to 31 December 2025
Outlet	Pune, Jaipur or Kochi
Item	Beverages, Bakery or Merchandise
Offer	The discount offered that day, in percent (0, 5, 10, 15, 20 or 25)

Column	Meaning
Revenue	Revenue of that item at that outlet on that day, in Rs thousand

The data is invented, so you can share it freely. Run Appendix A now. Then check that cafe.head() shows five rows like these:

```
        Date  Outlet         Item  Offer  Revenue
0 2025-01-01    Pune    Beverages      5    455.8
1 2025-01-01    Pune       Bakery      0    257.8
2 2025-01-01    Pune  Merchandise     20    156.1
3 2025-01-01  Jaipur    Beverages      0    338.1
4 2025-01-01  Jaipur       Bakery      0    207.1
```

1. A first look at any dataset

Before you draw a chart, answer four questions about the table. How big is it? What dates does it cover? What type is each column? Is anything missing? This takes one minute and saves you from charts built on data that is not what you thought. For example, a missing month would show up as a false dip in a trend, and a date stored as text would sort wrongly.

Code

```
cafe.shape
cafe.info()
cafe.describe()
cafe.isnull().sum()
cafe["Date"].min(), cafe["Date"].max()
```

Code	What it does
cafe.shape	Gives the size as (rows, columns). It is an attribute, so it has no brackets. Ours is (3285, 5): 365 days × 3 outlets × 3 items = 3,285 rows.
cafe.info()	Lists each column, how many values are filled (Non-Null Count) and its type (Dtype). Look for a Date column that is not a date type, or a Revenue column that is not a number. It takes no arguments here.
cafe.describe()	Summarises each numeric column: count, mean, standard deviation (std), minimum, quartiles (25%, 50%, 75%) and maximum. By default it shows only number and date columns. Always compare the maximum with the 75% value: a maximum far above it is a warning sign. Here the Revenue maximum is 3294.2 while 75% of rows are below 379.9.
cafe.isnull().sum()	isnull() marks every empty cell True, and sum() adds up the True values in each column, giving the number of missing values per column. All zeros means nothing is missing.
cafe["Date"].min() and .max()	The earliest and latest value of a column. Together they give the date range.

What it prints (shortened)

```
(3285, 5)

Date       0
Outlet     0
Item       0
Offer      0
Revenue    0

2025-01-01 to 2025-12-31
```

From these outputs you can say: the table has 3,285 rows, covers 1 January to 31 December 2025 and has no missing values. Write down these three facts; every report starts from them. Note the large gap between the Revenue maximum (3294.2) and the 75% value (379.9). You will come back to it in section 5.

Adding date parts: Month and Weekday

A Date column holds one value per day, which is too detailed for a trend. To group by month or by day of the week, add two helper columns using the .dt tools, which read the parts of a date.

Code

```
cafe["Month"] = cafe["Date"].dt.month
cafe["Weekday"] = cafe["Date"].dt.dayofweek
cafe.head(3)
```

Code	What it does
cafe["Month"] = ...	Creates a new column called Month. Assigning to a column name that does not exist adds it at the right of the table.
cafe["Date"].dt	Opens the date tools of a column. It works only on columns with a date type, which is why the type check in .info() matters.
.dt.month	Returns the month as a number, 1 for January to 12 for December.
.dt.dayofweek	Returns the day of the week as a number, 0 for Monday to 6 for Sunday. This is why the weekend is 5 and 6.
cafe.head(3)	Shows the first 3 rows (the default is 5), so you can check that the new columns look right.

What it prints

```
        Date Outlet         Item  Offer  Revenue  Month  Weekday
0 2025-01-01   Pune    Beverages      5    455.8      1        2
1 2025-01-01   Pune       Bakery      0    257.8      1        2
2 2025-01-01   Pune  Merchandise     20    156.1      1        2
```

YOUR TURN · A FIRST LOOK

Using the café data, answer in a Markdown cell: how many rows are there, what is the date range, and is anything missing? Then count how many rows each Outlet has with cafe["Outlet"].value_counts(), and say why the three numbers are equal. (Answers: Appendix B, 1.)

2. Patterns over time: the year and the week

Time is the first angle to look from, because sales rarely behave the same all year or all week. Two questions are enough to start: how does the total change from month to month, and does the day of the week matter? A line chart suits the first, because the line joins points that follow each other in time. A bar chart suits the second, because the seven days are separate groups to compare.

The monthly trend

Code

```
monthly = cafe.groupby("Month")["Revenue"].sum()
monthly.plot(marker="o")
```

Code	What it does
cafe.groupby("Month")	Splits the table into 12 groups, one per month. The argument is the column to split by.
["Revenue"]	Picks the column to calculate on inside each group.
.sum()	Adds up Revenue in each group. The result has one row per month. Use .mean() for an average instead of a total.
.plot(marker="o")	Draws a line chart from a Pandas Series. The index (the month numbers) goes on the x-axis and the values on the y-axis. marker="o" puts a dot on every data point so that you can see each month; without it you see only a line.

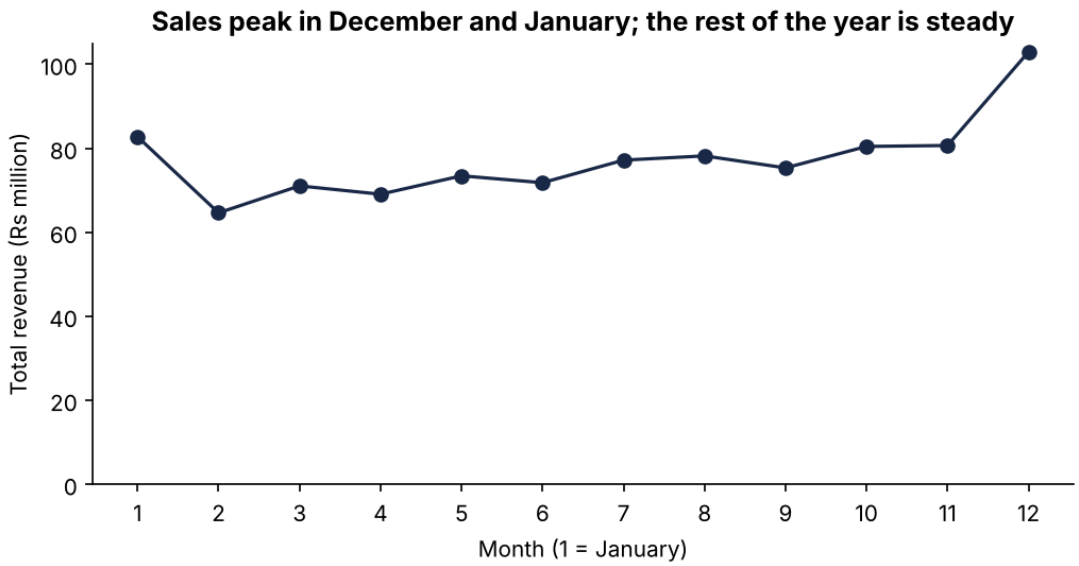


Figure 1. Total revenue for each month. (The title and axis labels are added with plt.title, plt.xlabel and plt.ylabel.)

Reading the chart: January is high, February is the lowest month, March to November rise slowly, and December is the highest by a clear margin. A jump needs an explanation, so form a guess and then test it. The café chain sells hot drinks, so a winter rise is a reasonable guess. January and December are the winter months in the data, and both are high. You can confirm it with the numbers rather than the eye: monthly.round(0) shows December at about 102.9 million against about 80 million in November. State the pattern, give the numbers, and say what could explain it, but say “could” until you have evidence, such as a calendar of events.

What monthly / 1000 prints (Rs million)

```
Month
1      82.7
2      64.6
3      71.0
4      69.1
5      73.4
6      71.8
7      77.2
8      78.2
9      75.3
10     80.4
11     80.7
12    102.9
```

The weekly pattern

Code

```
weekday_avg = cafe.groupby("Weekday")["Revenue"].mean()
plt.bar(range(7), weekday_avg.values)
plt.xticks(range(7), ["Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun"])
```

Code	What it does
groupby("Weekday")["Revenue"].mean()	The average Revenue per row for each day number 0 to 6. We use the average here, not the sum, because every weekday appears about the same number of times anyway, and an average is easier to read.
plt.bar(x, height)	Draws vertical bars. x is the positions of the bars, here range(7), which gives 0 to 6. height is the bar heights, here weekday_avg.values, the numbers without their index.
range(7)	The whole numbers 0, 1, 2, 3, 4, 5, 6, one for each day.
plt.xticks(positions, labels)	Sets the text under the bars. The first argument says where the labels go; the second is the list of names to show there. Without it the reader sees 0 to 6 and has to guess that 0 is Monday.

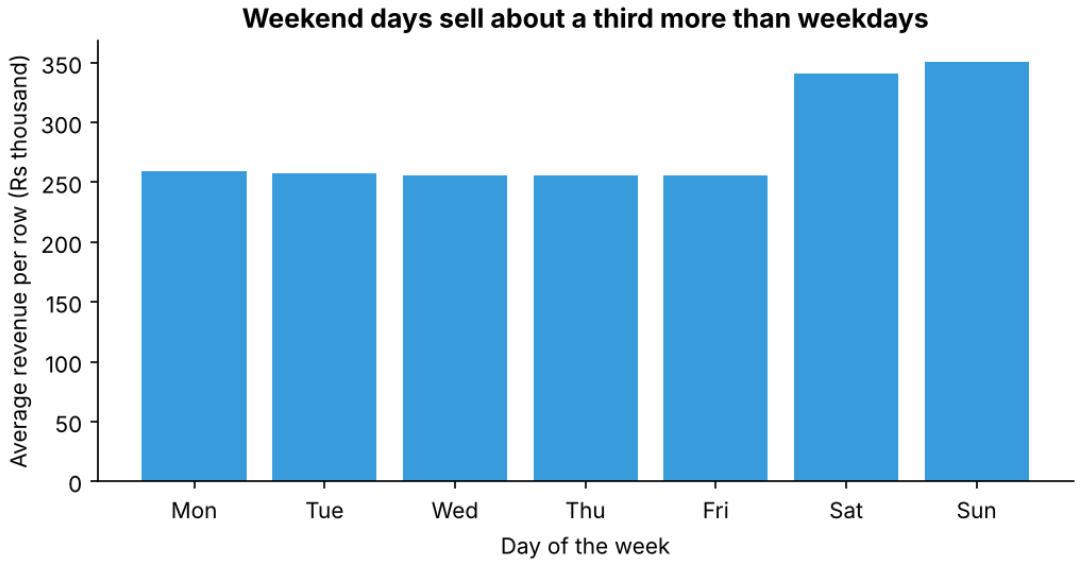


Figure 2. Average revenue per row for each day of the week.

What weekday_avg prints

```
Weekday
0    259.6
1    257.9
2    256.2
3    255.7
4    255.6
5    341.0
6    350.8
```

Saturday (341.0) and Sunday (350.8) are clearly higher than the five weekdays, which are all close to 256. The weekend average is about 346 against 257, which is roughly a third more. To work out “about how much”, divide the weekend average by the weekday average and subtract 1. Always give a number: “weekends sell more” is a claim, while “weekends sell about 35% more” is a finding.

YOUR TURN · TRENDS OVER TIME

1. Draw monthly total revenue for each Item as three lines on one chart. Hint: cafe.groupby(["Month", "Item"])["Revenue"].sum().unstack().plot(marker="o"). Which item grows most from February to December?

2. For Merchandise only, draw the average revenue for each weekday. Is the weekend lift similar to the one for all items? (Answers: Appendix B, 2 and 3.)

3. Two categories at once: pivot tables and heatmaps

A bar chart can compare one category, such as Outlet. The question “which outlet sells which item best” has two categories, so you need a grid. A pivot table puts one category in the rows and the other in the columns, with a calculated value in each cell. A heatmap then colours the cells, so the highest and lowest cells can be seen without reading every number.

Code

```
grid = cafe.pivot_table(index="Outlet", columns="Item", values="Revenue", aggfunc="mean")
sns.heatmap(grid, annot=True, fmt=".0f", cmap="Blues")
```

Code	What it does
pivot_table(index=, columns=, values=, aggfunc=)	Builds the grid. index is the column whose values become the rows. columns is the column whose values become the column headings. values is the number to calculate. aggfunc is how to combine the values that land in the same cell: "mean" for an average, "sum" for a total, "count" for a number of rows.
sns.heatmap(data)	Draws the grid as coloured squares. data is the pivot table. Darker means higher with the colour map we use.
annot=True	Writes the number inside each square. Without it you see colours only and cannot read the values.
fmt=".0f"	How to print the number: a float with 0 digits after the decimal point. ".1f" would show one decimal. It matters only when annot=True.
cmap="Blues"	The colour map: which colours go from low to high. "Blues" runs from white to dark blue, which is a good choice for amounts that only go up.

What grid prints

```
Item    Bakery  Beverages  Merchandise
Outlet
Jaipur   248.5      414.7        124.5
Kochi    261.8      437.7        131.5
Pune     287.9      488.3        145.9
```

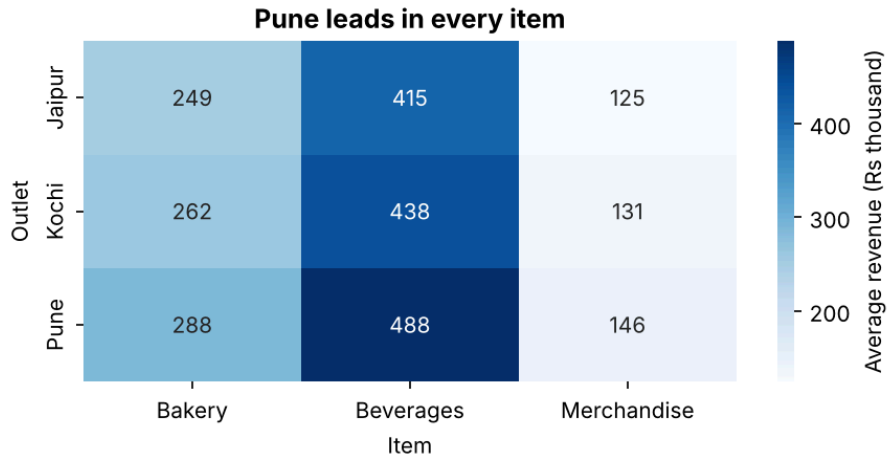


Figure 3. Average revenue per row, by outlet and item.

How to read a heatmap: first find the darkest and the lightest cell, then compare down each column and along each row. Here the darkest cell is Pune Beverages (488). Pune is the highest outlet in every column, and Kochi is second in every column, so the order of the outlets does not change with the item. Beverages is the highest item in every row, then Bakery, then Merchandise. When the order is the same everywhere you can say so in one sentence; when it changes, that change is usually the most interesting finding.

YOUR TURN · A DIFFERENT GRID

Build a pivot table with Month as the rows, Outlet as the columns and the average Revenue as the values. Draw it as a heatmap. Which month and outlet together give the highest cell? Is the outlet order the same in every month? (Answers: Appendix B, 4.)

4. Does a factor go with the result? The offer

Managers often ask whether a discount “works”. A first look uses the same tools: group by the factor and average the result. The chart below shows average revenue for each Offer level.

Code

```
offer_avg = cafe.groupby("Offer")["Revenue"].mean()
plt.bar(offer_avg.index.astype(str), offer_avg.values)
offer_avg.diff()
```

Code	What it does
groupby("Offer")["Revenue"].mean()	The average revenue for each of the six offer levels.
offer_avg.index.astype(str)	The offer levels are numbers, and a bar chart would place them as numeric positions with uneven spacing. astype(str) turns them into text labels, so each bar gets equal space. The argument str means “convert to text”.
offer_avg.diff()	The difference between each value and the one before it. It answers “how much did the average rise at each step?”. The first row has no earlier row, so it shows NaN (not a number).

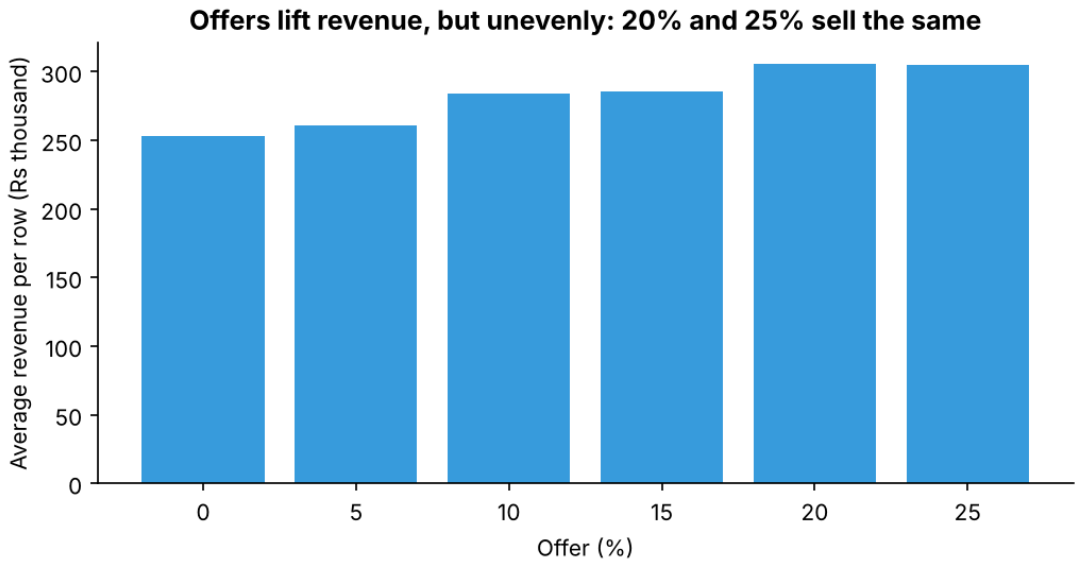


Figure 4. Average revenue for each offer level.

What it prints

```
Offer
0     253.2
5     260.7
```

```
10    284.0
15    285.6
20    305.7
25    304.7

Step changes (diff):
Offer
0      NaN
5      7.5
10    23.3
15     1.6
20    20.2
25    -1.1
```

Higher offers do go with higher revenue: the average moves from 253 at 0% to about 305 at 20% and 25%. But the steps are not equal. Going from 10% to 15% adds only 1.6, from 15% to 20% adds 20.2, and from 20% to 25% adds nothing. Two cautions follow. First, “goes with” is not “causes”: this table cannot tell you that the offer made the extra sales, only that the two moved together. Second, an uneven step like 10% to 15% can come from other things that differ between the groups, such as how many weekend days or which items fall in each group. Say what you see and name the limit in one line.

YOUR TURN · THE OFFER, FOR ONE ITEM

Repeat the offer chart for Beverages only. Filter first with cafe[cafe["Item"] == "Beverages"], then group by Offer. Is the rise steady here? Compare the step sizes with .diff(). (Answers: Appendix B, 5.)

5. Outliers: find them, look at them, then decide

An outlier is a value far away from the others. It matters for two reasons. If it is a mistake, such as a number typed with an extra digit, it distorts every average and total you calculate. If it is real, such as an unusually busy day, deleting it throws away true information. So the rule is always the same: find it, look at it, decide, and write down what you did.

Reading a box plot

A box plot shows the spread of a number for each group. The box holds the middle half of the values, from the 25% line to the 75% line, and the line inside the box is the median, the middle value. The whiskers reach to the furthest values that are not far from the box. Roughly, “not far” means within one and a half times the box height. Every dot beyond the whiskers is flagged as unusual. Flagged does not mean wrong: it means “look here”.

Code

```
fig = px.box(cafe, x="Item", y="Revenue", hover_data=["Date", "Outlet"])
fig.show()
```

Code	What it does
px.box(data, x=, y=, hover_data=)	Draws one box plot per value of x. data is the table. x is the column that makes one box per value (here three boxes, one per Item). y is the number whose spread is shown. hover_data is a list of extra columns to show in the box that appears when you point at a dot; here the date and outlet, so each unusual dot can be identified.

Code	What it does
fig.show()	Displays the chart in the notebook. It is interactive: hover to see values, drag to zoom, double-click to reset.

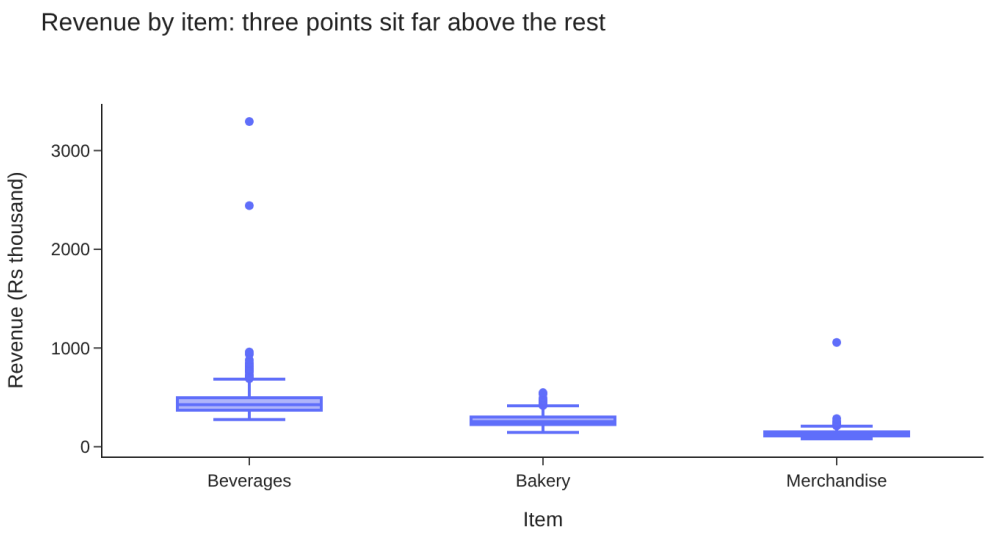


Figure 5. Revenue by item. Three dots lie far above everything else.

Three dots are far above the others: two in Beverages (about 3,294 and 2,442) and one in Merchandise (about 1,056). Hover over each to read its date, outlet and value. Many more dots sit just above the whiskers, between about 600 and 950 for Beverages. These are different. They are the busiest days, and the table below shows that they fall on winter weekends with high offers, which is where our earlier charts said sales should be high. They are normal, only large.

The three far dots, and the busiest normal days

```
           Date  Outlet         Item  Revenue
300  2025-02-03  Jaipur    Beverages   2441.6
1800 2025-07-20    Pune    Beverages   3294.2
2900 2025-11-19    Pune  Merchandise   1055.6

Busiest normal Beverages days:
           Date Outlet  Offer  Revenue
3249 2025-12-28   Pune     20    959.5
3051 2025-12-06   Pune     25    935.8
3123 2025-12-14   Pune     20    877.2
```

Investigate before you delete

Ask three questions of every far dot. Is it possible? A single beverage line selling Rs 3.3 million in a day, when the usual is under Rs 0.5 million, is not. Is there a cause in the data? A normal Saturday in December can explain a high value; a quiet Sunday in July cannot. Can it be checked with the source? In real work you would ask the outlet manager. Only after this do you remove it, and you write the reason in your notes. Removing a value because it is “too high” spoils the analysis; removing it because it is shown to be an error is good practice.

Removing the errors with a rule

Code

```
item_mean = cafe.groupby("Item")["Revenue"].transform("mean")
clean = cafe[cafe["Revenue"] <= 3 * item_mean]
len(cafe) - len(clean)
```

Code	What it does
groupby("Item")["Revenue"].transform("mean")	Calculates the mean of each Item, but returns one value for every row of the original table, so the result is as long as cafe. Each row gets the average of its own Item. (A plain .mean() would give only three numbers.) The argument "mean" says which calculation to run.
3 * item_mean	The cut-off for each row: three times its item's average. This rule says a row is suspect if it is more than three times the usual for its own item.
cafe[condition]	Keeps only the rows where the condition is True. Here: rows at or below the cut-off. The result is saved as clean, and cafe is not changed.
len(cafe) - len(clean)	len() counts rows. The difference is how many rows were removed. Always check it equals the number you meant to remove.

What it prints

```
3
```

Exactly 3 rows were removed, the three far dots, and the table has 3,282 rows left. Checking the count matters: a cut-off that removes 30 rows would be removing real days. Why compare with the item's own average instead of one overall average? Because a Rs 900 beverage day is normal while a Rs 900 merchandise day is not. A rule must compare like with like.

> **YOUR TURN · A RULE THAT IS TOO BLUNT**
>
> Repeat the rule with Outlet instead of Item: group by "Outlet", keep rows at or below 3 times the outlet mean, and count the rows removed. Is the count 3? Look at the extra rows removed and say whether they are errors. What does this teach you about choosing the group for the rule? (Answers: Appendix B, 6.)

6. Interactive charts with Plotly Express

The charts so far are pictures. Plotly Express makes charts that people can explore: pointing at a mark shows its exact values, dragging zooms in, clicking a legend name hides or shows that line, and double-clicking resets the view. This matters for a manager who wants to check one number without asking you to redraw the chart. From here on we use the clean table.

Plotly Express draws from a table, so first prepare the summary table, then pass it to the chart function. Four charts, four calls:

Code

```
# 1. Line chart: monthly revenue for each outlet
```

```
monthly = clean.groupby(["Month", "Outlet"], as_index=False)["Revenue"].sum()
fig_a = px.line(monthly, x="Month", y="Revenue", color="Outlet", markers=True)

# 2. Bar chart: total revenue for each item
by_item = clean.groupby("Item", as_index=False)["Revenue"].sum()
fig_b = px.bar(by_item, x="Item", y="Revenue")

# 3. Heatmap: average revenue by outlet and item
grid = clean.pivot_table(index="Outlet", columns="Item", values="Revenue", aggfunc="mean")
fig_c = px.imshow(grid, text_auto=".0f", color_continuous_scale="Blues")

# 4. Bar chart: average revenue for each offer level
by_offer = clean.groupby("Offer", as_index=False)["Revenue"].mean()
fig_d = px.bar(by_offer, x="Offer", y="Revenue")

fig_a.show()
```

Code	What it does
groupby(["Month", "Outlet"], as_index=False)	Groups by two columns at once, so each month has one row per outlet. as_index=False keeps Month and Outlet as ordinary columns instead of turning them into the row labels. Plotly Express needs them as columns, because x= and color= take column names.
px.line(data, x=, y=, color=, markers=)	A line chart. x and y are column names. color="Outlet" draws one line per outlet, each in its own colour with a legend you can click. markers=True puts a dot on every point.
px.bar(data, x=, y=)	A bar chart from a summary table. It does not add values up for you, so give it a table that already has one row per bar.
px.imshow(grid, text_auto=, color_continuous_scale=)	Draws a grid of numbers as a heatmap. grid is the pivot table. text_auto=".0f" prints each value with no decimals inside its square. color_continuous_scale="Blues" sets the colours, as cmap did in Seaborn.
fig_a.show()	Displays one chart. Each chart is stored in a variable (fig_a to fig_d) so that we can reuse it.

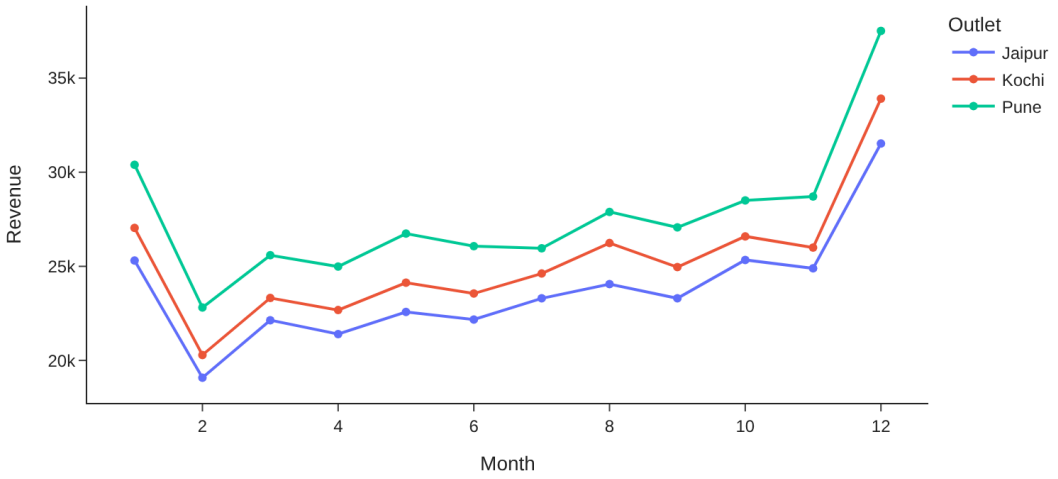


Figure 6. fig_a: monthly revenue for each outlet. In the notebook you can hover on any point and click an outlet name in the legend.

Try the four interactions on fig_a: hover on the December point of Pune, drag a box around March to May to zoom, click “Kochi” in the legend to hide it, and double-click to reset. In a report for others, tell them these actions exist; people do not try them unless told.

7. One dashboard from several charts

A dashboard puts several charts on one page, so the reader sees the whole picture without scrolling through a notebook. Plotly builds a dashboard by making an empty grid with make_subplots and copying the marks (called traces) of each chart into a cell. The long block below is one cell, divided into numbered sections; read the comments first.

Code

```
# 1. Import the tool that makes a grid of charts
from plotly.subplots import make_subplots

# 2. Make an empty 2 by 2 grid, with a title for each cell
dash = make_subplots(rows=2, cols=2, subplot_titles=(
    "Monthly revenue by outlet", "Total revenue by item",
    "Average revenue: outlet vs item", "Average revenue by offer (%)"))

# 3. Copy the marks of each chart into its cell
for trace in fig_a.data: dash.add_trace(trace, row=1, col=1)
for trace in fig_b.data: dash.add_trace(trace, row=1, col=2)
for trace in fig_c.data: dash.add_trace(trace, row=2, col=1)
for trace in fig_d.data: dash.add_trace(trace, row=2, col=2)

# 4. Set the heatmap colours again and hide its colour bar, then set the size, the title and the
look
dash.update_coloraxes(colorscale="Blues", showscale=False)
dash.update_layout(height=700, title_text="Cafe Sales Dashboard", template="simple_white")

# 5. Label the axes of two panels, with units
dash.update_xaxes(title_text="Month (1 = January)", row=1, col=1)
dash.update_yaxes(title_text="Revenue (Rs thousand)", row=1, col=1)
dash.update_xaxes(title_text="Offer (%)", row=2, col=2)
dash.update_yaxes(title_text="Average revenue (Rs thousand)", row=2, col=2)

# 6. Show it, then save it as a web page
dash.show()
dash.write_html("cafe_dashboard.html")
```

Code	What it does
make_subplots(rows=, cols=, subplot_titles=)	Makes an empty grid. rows and cols give its shape (2 and 2 make four cells). subplot_titles is a tuple of four titles, filled left to right, top to bottom.
fig_a.data	The list of traces (the lines, bars or squares) that make up chart fig_a. We loop over it with for.
dash.add_trace(trace, row=, col=)	Puts one trace into one cell of the grid. row and col count from 1, not 0, so row=1, col=2 is the top right.
dash.update_coloraxes(colorscale=, showscale=)	Changes the colour settings of the heatmap. Copying a chart into a grid loses its colour scale, so colorscale="Blues" sets it again; without it the squares come out in purple and yellow. showscale=False hides the colour bar, because the numbers are already printed in the squares and the bar would take space.
dash.update_layout(height=, title_text=, template=)	Changes the whole figure. height is in pixels. title_text is the title over everything. template="simple_white" gives a clean white look with light lines.
dash.update_xaxes(title_text=, row=, col=)	Sets the x-axis label of one cell. update_yaxes does the same for the y-axis. row and col pick the cell. Include the unit in the label.
dash.show()	Shows the dashboard in the notebook.
dash.write_html("cafe_dashboard.html")	Saves the dashboard as a web page. The argument is the file name, which is created in the notebook's folder. The file has everything inside it, including the interactive code, so it is large (a few MB), and it opens in any browser with no Python.



Figure 7. The finished dashboard. The saved web page is interactive: hover, zoom and legend clicks all work.

After saving, open the file from your folder by double-clicking it. Check that hovering, zooming and clicking a legend name still work, because a dashboard that only looks right is not finished. If a panel looks wrong, fix it in the notebook and save again.

Choosing the chart that tells the story

You do not have to use these four charts. A panel earns its place by answering a question quickly. If you want to show how each outlet's December compares with the others, a heatmap of Month by Outlet may tell the story better than four lines. Replace a panel when you can say in one sentence why the new chart is clearer, and say it in your notes.

> **YOUR TURN · BUILD YOUR OWN**
>
> Make a dashboard of two charts side by side (rows=1, cols=2): total revenue by Outlet, and monthly revenue for Beverages only. Give both axes titles with units and save it as an HTML file. Open it and check hover and zoom. (Answers: Appendix B, 7.)

8. Turning the dashboard into an app with Streamlit

The dashboard in section 7 is a saved page. Everyone who opens it sees the same four charts, and the only choices they have are hover, zoom and the legend. That is fine for a report, but a manager often asks "can I see only Pune?" or "what about just November and December?". A saved page cannot answer without you redrawing it. Streamlit is a Python library that turns a script into a small web app. You add filters, and every chart and number is recalculated when someone changes a filter. You do not need to know HTML or JavaScript; you write Python, and Streamlit builds the page.

	Saved page (write_html)	Streamlit app
What the reader can do	Hover, zoom, click the legend	All of that, plus choose outlet, item and months with filters
How the reader opens it	Double-click the file; no Python needed	Python must be running the app; the reader opens a web address
How you share it	Send the file	Run it on your computer or on a server, and share the address
When to use it	A finished report that will not change	A tool the reader explores themselves

Step 1: install Streamlit and save the clean data

A Streamlit app is a separate Python file, not a notebook cell. It needs the data, so we save the clean table from the notebook into a file the app can read. This also means the app does not repeat the cleaning work.

Run these in your notebook

```
%pip install streamlit
clean.to_csv("cafe_clean.csv", index=False)
```

Code	What it does
%pip install streamlit	Installs the library into the Python that the notebook uses. The % makes it run in the notebook. You need it only once. If you prefer a terminal, run pip install streamlit there instead.
clean.to_csv("cafe_clean.csv", index=False)	Saves the table as a CSV file. The first argument is the file name; it is created in the notebook's folder. index=False leaves out the row numbers, which would otherwise be saved as an extra unnamed column.

Step 2: write the app

Create a new text file called app.py in the same folder as cafe_clean.csv. (In Jupyter: New, then Text File, then rename it. Or put %%writefile app.py as the first line of a notebook cell holding the code below.) The code is one file, divided into numbered sections. Streamlit runs the file from top to bottom every time a filter changes, which is the most important idea to understand: you do not write "when the user clicks, do this"; you write the page as if you were drawing it once, and Streamlit draws it again after every click.

app.py

```
# 1. Libraries
import pandas as pd
import plotly.express as px
import streamlit as st

# 2. Page settings: the browser-tab title and a wide layout
st.set_page_config(page_title="Cafe Sales Dashboard", layout="wide")


# 3. Load the clean data once; Streamlit remembers it between clicks
@st.cache_data
def load_data():
    return pd.read_csv("cafe_clean.csv", parse_dates=["Date"])
```

```
clean = load_data()

# 4. Title and a short line of help
st.title("Cafe Sales Dashboard")
st.write("Use the filters on the left. Every chart and number updates at once.")

# 5. Filters in the sidebar
st.sidebar.header("Filters")
outlets = st.sidebar.multiselect("Outlet", sorted(clean["Outlet"].unique()),
                                 default=sorted(clean["Outlet"].unique()))
items = st.sidebar.multiselect("Item", sorted(clean["Item"].unique()),
                               default=sorted(clean["Item"].unique()))
first_month, last_month = st.sidebar.slider("Months (1 = January)", 1, 12, (1, 12))

# 6. Keep only the rows that match the filters
view = clean[clean["Outlet"].isin(outlets)
             & clean["Item"].isin(items)
             & clean["Month"].between(first_month, last_month)]

# 7. If the filters leave nothing, say so and stop
if view.empty:
    st.warning("No rows match these filters. Choose at least one outlet and one item.")
    st.stop()

# 8. Three headline numbers
col1, col2, col3 = st.columns(3)
col1.metric("Total revenue (Rs million)", f"{view['Revenue'].sum() / 1000:,.1f}")
col2.metric("Average per row (Rs thousand)", f"{view['Revenue'].mean():,.0f}")
col3.metric("Rows shown", f"{len(view):,}")

# 9. Four charts in a 2 by 2 layout
monthly = view.groupby(["Month", "Outlet"], as_index=False)["Revenue"].sum()
fig_a = px.line(monthly, x="Month", y="Revenue", color="Outlet", markers=True,
                title="Monthly revenue by outlet",
                labels={"Revenue": "Revenue (Rs thousand)"})

by_item = view.groupby("Item", as_index=False)["Revenue"].sum()
fig_b = px.bar(by_item, x="Item", y="Revenue", title="Total revenue by item",
               labels={"Revenue": "Revenue (Rs thousand)"})

grid = view.pivot_table(index="Outlet", columns="Item", values="Revenue", aggfunc="mean")
fig_c = px.imshow(grid, text_auto=".0f", color_continuous_scale="Blues",
                  title="Average revenue: outlet vs item")

by_offer = view.groupby("Offer", as_index=False)["Revenue"].mean()
fig_d = px.bar(by_offer, x="Offer", y="Revenue", title="Average revenue by offer (%)",
               labels={"Revenue": "Average revenue (Rs thousand)", "Offer": "Offer (%)"})

left, right = st.columns(2)
left.plotly_chart(fig_a)
right.plotly_chart(fig_b)
left.plotly_chart(fig_c)
right.plotly_chart(fig_d)

# 10. The rows behind the charts, hidden until the reader asks
with st.expander("See the data behind the charts"):
    st.dataframe(view)
```

Code	What it does
st.set_page_config(page_title=, layout=)	Sets how the page looks. It must be the first Streamlit call. page_title is the text on the browser tab. layout="wide" uses the full width of the screen instead of a narrow column, which suits a dashboard.

Code	What it does
@st.cache_data	Placed above a function. Streamlit remembers what the function returned, so the file is read once and not after every click. Without it the app still works but reloads the data each time and feels slow on bigger files.
pd.read_csv(path, parse_dates=["Date"])	Reads the CSV file. path is the file name. parse_dates is a list of columns to turn into real dates; a CSV stores dates as text, so without it Date would be text.
st.title(text) and st.write(text)	Show a large heading and an ordinary line of text on the page.
st.sidebar.header(text)	Puts a heading in the sidebar, the panel on the left. Anything starting with st.sidebar appears there.
st.sidebar.multiselect(label, options, default=)	A box where the reader picks one or more values. label is the text above it. options is the list to choose from; sorted(clean["Outlet"].unique()) gives the three outlet names in order. default is what is selected at the start; here all of them. It returns the chosen values as a list.
st.sidebar.slider(label, min, max, value)	A slider. min and max are the ends (1 and 12). value is the start position; giving a pair such as (1, 12) makes a range slider with two handles. It returns the two chosen numbers, which we unpack into first_month and last_month.
clean["Outlet"].isin(outlets)	True for each row whose outlet is in the chosen list. The & between conditions means “and”: a row is kept only if all three conditions are true. Each condition is in its own line inside brackets, so the code reads clearly.
clean["Month"].between(first, last)	True for rows whose month is from first to last, both included.
view.empty	True when the filtered table has no rows, which happens if the reader removes every outlet. Without this check the charts would fail with an error message.
st.warning(text) and st.stop()	Show a yellow message and end the script there, so the code that draws the charts does not run on empty data.
st.columns(3)	Splits the page into three side-by-side columns and returns them. col1.metric(...) puts a value in the first one. With st.columns(2) we make a left and a right side for the charts.
col1.metric(label, value)	Shows a big number with a small label above it. value should be text; the f-string f"{x:,.1f}" turns a number into text with commas and one decimal place.
px.line(...), px.bar(...), px.imshow(...)	The same Plotly Express calls as in section 6. Two new arguments appear: title= puts a title on the chart, and labels={"Revenue": "Revenue (Rs thousand)"} renames an axis, so the unit is shown.
left.plotly_chart(fig)	Draws a Plotly chart in the left column. The chart is as interactive as before: hover, zoom and the legend still work.

Code	What it does
st.expander(text)	A closed box with a title; clicking it opens it. We use it with with to put st.dataframe(view) inside, so the table of rows stays out of the way until someone wants it.

Step 3: run the app

Open a terminal in the folder that holds app.py and cafe_clean.csv. On Windows use Anaconda Prompt or Command Prompt; on a Mac use Terminal. Then run:

In the terminal

```
streamlit run app.py
```

Streamlit starts a small server on your computer and opens the app in your browser, at an address like http://localhost:8501. “localhost” means your own computer, so only you can see it. Try the filters: remove Jaipur and Kochi, and every chart and number shows only Pune. Drag the month slider to 11 and 12 and the totals fall to the last two months. Remove every outlet and the yellow warning appears. To stop the app, go back to the terminal and press Ctrl and C together.

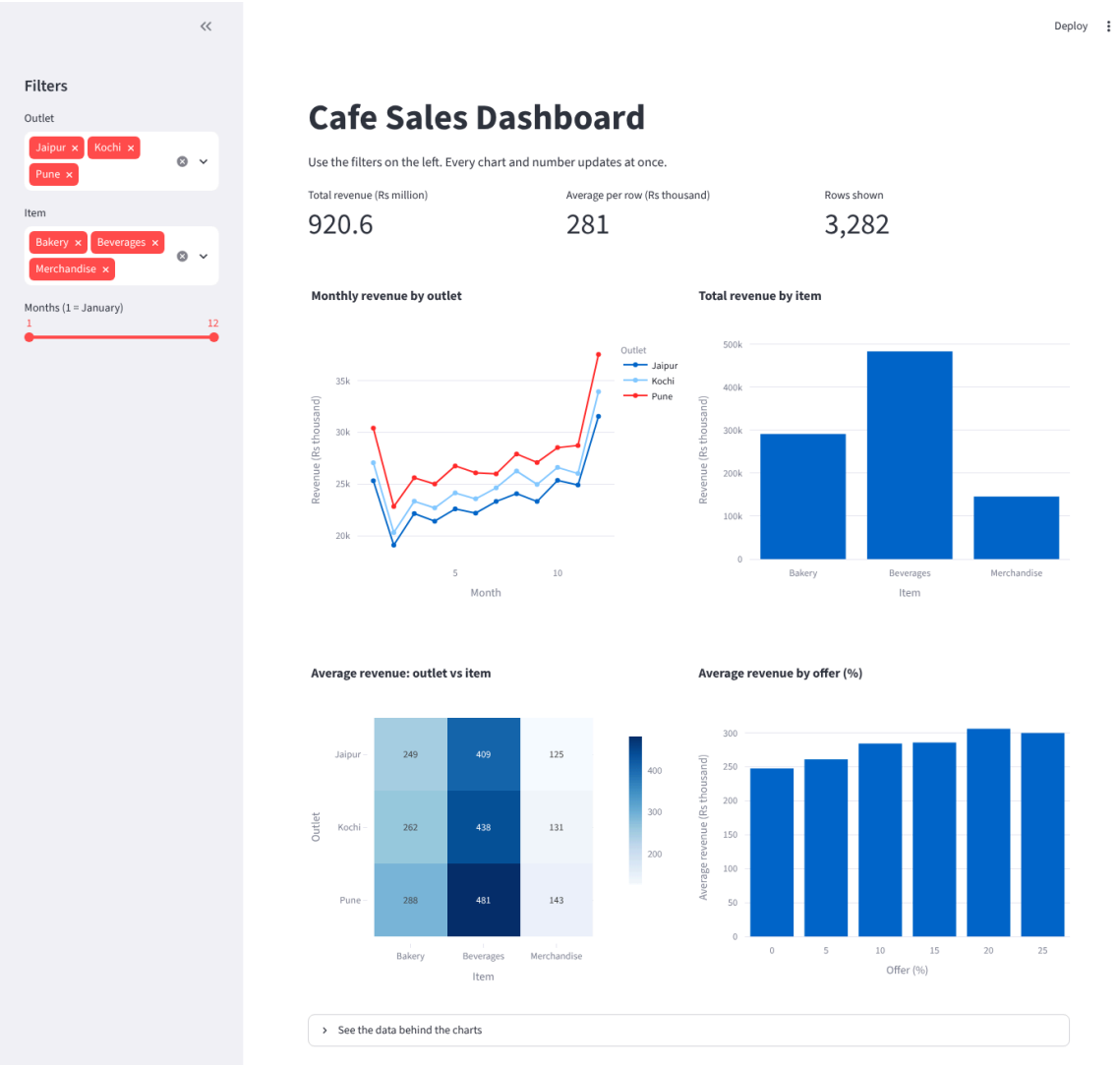


Figure 8. The app with all filters on. The numbers at the top and the four charts all change when a filter changes.

Check your work against the numbers: with all filters on, the app shows 3,282 rows and a total of 920.6 million. If you choose only Pune and months 11 and 12, it shows 182 rows and a total of 66.2 million.

Optional: running it in Google Colab

Colab runs on a computer in Google's cloud, not yours, so "localhost" is not reachable from your browser. To see the app you need a tunnel, which is a tool that gives the app a temporary public web address. One free tunnel tool, LocalTunnel, is run with npx. Run the cells in your Colab notebook that build clean and save cafe_clean.csv, then:

Cell 1: install, and print this Colab machine's public address

```
%pip install streamlit
!wget -q -O - ipv4.icanhazip.com
```

Cell 2: save the app file (the first line must be exactly this)

```
%%writefile app.py
(paste the code of app.py here)
```

Cell 3: start the app and the tunnel

```
!streamlit run app.py & npx localtunnel --port 8501
```

Cell 3 prints a link ending in loca.lt. Open it. The page may ask for a password; it is the address printed by Cell 1. The app runs only while this cell is running and the Colab session is open, and the tunnel is a free third-party service that sometimes fails or is slow. If it does not work, run the app on your own computer as in Step 3. Colab is a good place to learn and test, not a place to host an app that people depend on.

> **YOUR TURN · ADD TO THE APP**
>
> 1. Add a slider to the sidebar for Offer (from 0 to 25, start with the full range) and make the filter keep only rows in that range.
>
> 2. Add a fourth headline number, "Best outlet", showing the name of the outlet with the highest total revenue in the current view. Hint: view.groupby("Outlet")["Revenue"].sum().idxmax().
>
> Run the app and check that the number changes when you move the offer slider. (Answers: Appendix B, 8.)

9. Writing for a manager

The last step is where many analyses fail. A manager does not want code or chart settings. They want to know what you found, whether to trust it, and what to do. A one-page report with four parts works well, and each part has a job.

Part	What it contains	The test
Title and purpose	One line: what the report covers and which period	A stranger knows what this is in five seconds
Three key findings	Three sentences, each with one number from your charts	Each can be checked against a chart
Data quality note	Which unusual values you found and what you did about them	The reader knows how far to trust the numbers

Part	What it contains	The test
Two recommendations	Two actions the manager can take, each tied to a finding	Each starts with a verb and could be done next week

An example

Here is a short report on the café data. Notice the plain words, the numbers, and that each recommendation follows from a finding.

EXAMPLE REPORT

Café sales in 2025: what the data shows. This report covers daily sales at our three outlets from January to December 2025.

Findings.

- December is our best month: 102.9 million, about 28% above November (80.7 million), and January is also high.
- Weekends sell about 35% more than weekdays (about 346 against 257 Rs thousand per line).
- Pune leads at every item: its Beverages average is 488, against 438 in Kochi and 415 in Jaipur.

Data quality. Three entries were far larger than normal (for example Rs 3.3 million of beverages in one day in July) and look like typing errors. We removed them. Busy winter weekends were kept because they are real.

Recommendations.

- Staff and stock for the winter weekend peak: plan extra stock from late November.
- Ask Jaipur and Kochi what Pune does differently on Beverages, and test it for one month.

Compare this with a weak report: “The dataset exhibits a seasonal uplift with a statistically notable weekend effect.” It may be true, but the manager cannot act on it, and it has no numbers. Three habits make the difference: use the words your reader uses, put a number in every finding, and finish with an action.

YOUR TURN · WRITE IT YOURSELF

Using your own charts from the sections above, write a report on the café data in your own words, with the four parts. Keep it to one page. Then read it aloud once and replace every word you would not say to a friend. (A sample is in Appendix B, 9; try yours first.)

Appendix A: Practice-data cell

Run this once at the top of your notebook. It is one cell, divided into numbered sections. The seed makes the random numbers the same for everyone, so your outputs match this guide.

Code

```
# 1. Libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import plotly.express as px

# 2. Build the practice data: daily sales of a cafe chain, three outlets, one year
np.random.seed(21)
dates = pd.date_range("2025-01-01", "2025-12-31", freq="D")
outlets = ["Pune", "Jaipur", "Kochi"]
items = ["Beverages", "Bakery", "Merchandise"]

cafe = pd.DataFrame(
    [(d, o, i) for d in dates for o in outlets for i in items],
    columns=["Date", "Outlet", "Item"],
)

# 3. Add the factors that shape sales
base = {"Beverages": 300, "Bakery": 180, "Merchandise": 90}
n = len(cafe)
cafe["Offer"] = np.random.choice([0, 5, 10, 15, 20, 25], n)
growth = 1 + cafe["Date"].dt.dayofyear / 365 * 0.25
weekend = np.where(cafe["Date"].dt.dayofweek >= 5, 1.35, 1.0)
winter = np.where(cafe["Date"].dt.month.isin([12, 1]), 1.25, 1.0)
outlet_factor = cafe["Outlet"].map({"Pune": 1.1, "Jaipur": 0.95, "Kochi": 1.0})

# 4. Revenue = base x growth x weekend x winter x outlet x offer x random noise
cafe["Revenue"] = (cafe["Item"].map(base) * growth * weekend * winter * outlet_factor
                   * (1 + cafe["Offer"] / 100) * np.random.normal(1, 0.07, n)).round(1)

# 5. Hide a few unusual values (data-entry slips) for you to find
cafe.loc[[300, 1800, 2900], "Revenue"] *= 7

cafe.head()
```

Code	What it does
import ... as ...	Loads a library. as np, as pd, as plt, as sns and as px give the usual short names used throughout this guide.
np.random.seed(21)	Fixes the random number generator, so every run produces the same table. The number 21 is the seed; a different number would give a different table.
pd.date_range(start, end, freq="D")	Makes a list of dates from start to end. freq="D" means one per day.
pd.DataFrame(rows, columns=)	Builds a table. rows is a list of tuples (one per row) and columns names them. The list comprehension makes every combination of day, outlet and item.
np.random.choice(list, n)	Picks n random values from the list. Here it gives each row a random offer.

Code	What it does
np.where(condition, a, b)	Gives a where the condition is true and b where it is not. Here: a weekend boost, and a winter boost for December and January.
.map({...})	Replaces each value with its partner in a dictionary. Here it turns an item or an outlet name into its sales factor.
np.random.normal(1, 0.07, n)	n random numbers centred on 1 with a spread of 0.07, so each revenue varies by about 7% around its expected value.
.round(1)	Rounds to 1 decimal place.
cafe.loc[[300, 1800, 2900], "Revenue"] *= 7	Selects three rows by their position label and the Revenue column, and multiplies them by 7. This creates the three hidden errors.

Appendix B: Answers to the “Your turn” boxes

Try each exercise first. The code below is one cell with numbered sections that match the boxes. The results come from running it on the practice data, and your numbers should match. Written answers are given in plain sentences; yours should be in your own words.

Code

```
# 1. A first look
print(cafe.shape)
print(cafe["Date"].min(), cafe["Date"].max())
print(cafe.isnull().sum().sum())
print(cafe["Outlet"].value_counts())

# 2. Monthly revenue for each item, as three lines
by_item_month = cafe.groupby(["Month", "Item"])["Revenue"].sum().unstack()
by_item_month.plot(marker="o")
(by_item_month.loc[12] / by_item_month.loc[2] - 1) * 100

# 3. Merchandise only: average revenue per weekday
merch = cafe[cafe["Item"] == "Merchandise"].groupby("Weekday")["Revenue"].mean()
merch.plot(kind="bar")
merch[[5, 6]].mean() / merch[[0, 1, 2, 3, 4]].mean() - 1

# 4. Month by outlet heatmap
grid2 = cafe.pivot_table(index="Month", columns="Outlet", values="Revenue", aggfunc="mean")
sns.heatmap(grid2, annot=True, fmt=".0f", cmap="Blues")

# 5. The offer for Beverages only
bev = cafe[cafe["Item"] == "Beverages"].groupby("Offer")["Revenue"].mean()
bev.plot(kind="bar")
bev.diff()

# 6. The rule grouped by outlet (too blunt)
outlet_mean = cafe.groupby("Outlet")["Revenue"].transform("mean")
too_many = cafe[cafe["Revenue"] > 3 * outlet_mean]
len(too_many)
too_many.sort_values("Revenue", ascending=False).head(8)

# 7. A two-chart dashboard
from plotly.subplots import make_subplots
by_outlet = clean.groupby("Outlet", as_index=False)["Revenue"].sum()
bev_month = clean[clean["Item"] == "Beverages"].groupby("Month", as_index=False)["Revenue"].sum()
fig_o = px.bar(by_outlet, x="Outlet", y="Revenue")
```

```
fig_m = px.line(bev_month, x="Month", y="Revenue", markers=True)
two = make_subplots(rows=1, cols=2, subplot_titles=("Total revenue by outlet", "Beverages revenue
by month"))
for t in fig_o.data: two.add_trace(t, row=1, col=1)
for t in fig_m.data: two.add_trace(t, row=1, col=2)
two.update_xaxes(title_text="Outlet", row=1, col=1)
two.update_yaxes(title_text="Revenue (Rs thousand)", row=1, col=1)
two.update_xaxes(title_text="Month (1 = January)", row=1, col=2)
two.update_yaxes(title_text="Revenue (Rs thousand)", row=1, col=2)
two.update_layout(height=450, title_text="Outlets and beverages", template="simple_white",
showlegend=False)
two.show()
two.write_html("outlets_and_beverages.html")
```

1. A first look

```
(3285, 7)
2025-01-01 2025-12-31
0
```

The table has 3,285 rows, covers 1 January to 31 December 2025, and nothing is missing. Each outlet has 1,095 rows (365 days × 3 items), so the three counts are equal because every outlet sold every item every day.

2. Monthly revenue for each item (Rs million)

```
Item   Bakery  Beverages  Merchandise
Month
1        26.6       43.0         13.1
2        19.8       34.9          9.9
6        22.5       38.0         11.3
11       24.9       42.2         13.6
12       32.1       54.5         16.3

Percent rise from February to December:
Item
Bakery         63.0
Beverages      56.0
Merchandise    64.0
```

All three items follow the same shape: low in February, high in December. Merchandise grows most (64%), but Bakery (63%) and Beverages (56%) are close, so no item is driving the change on its own.

3. Merchandise only, average revenue per weekday

```
Weekday
0    120.0
1    119.9
2    125.4
3    122.1
4    122.0
5    165.4
6    163.2

Weekend lift: 0.348
```

The weekend lift for Merchandise is about 35%, almost the same as for all items together (about 35%). The weekend effect is not specific to one item.

4. Month by outlet heatmap

```
Outlet  Jaipur  Kochi   Pune
Month
1        272.0  291.0  327.0
```

```
2        256.0  242.0  272.0
3        238.0  251.0  275.0
4        238.0  252.0  278.0
5        243.0  259.0  287.0
6        246.0  262.0  290.0
7        251.0  265.0  315.0
8        259.0  282.0  300.0
9        259.0  277.0  301.0
10       272.0  286.0  306.0
11       277.0  289.0  331.0
12       339.0  365.0  403.0

Highest cell: (12, 'Pune') 403.3
```

The highest cell is Pune in December (about 403). The outlet order is Pune, Kochi, Jaipur in most months, which means the order is stable, but in February Kochi is a little below Jaipur, so check each month before saying “every”.

5. The offer, for Beverages only

```
Offer
0     418.3
5     411.6
10    435.7
15    445.3
20    469.9
25    508.8

Step changes:
Offer
0      NaN
5     -6.7
10    24.1
15     9.6
20    24.6
25    38.9
```

For Beverages the rise is more regular than for all items: after a small dip at 5%, each step adds revenue, and the 25% level is highest (about 509 against 418 at 0%). The steps still differ (from 9.6 to 38.9), so the rise is not perfectly steady. As before, this shows the two move together, not that the offer caused it.

6. The rule grouped by outlet

```
7
           Date  Outlet         Item  Revenue
1800 2025-07-20    Pune    Beverages   3294.2
300  2025-02-03  Jaipur    Beverages   2441.6
2900 2025-11-19    Pune  Merchandise   1055.6
3249 2025-12-28    Pune    Beverages    959.5
3051 2025-12-06    Pune    Beverages    935.8
3246 2025-12-27   Kochi    Beverages    849.7
3252 2025-12-28  Jaipur    Beverages    811.5
```

The count is 7, not 3. The three errors are still caught, but four more rows are removed (all Beverages rows from December, at Pune, Kochi and Jaipur, between about 810 and 960). Those are real busy winter days, not errors. The lesson: compare each value with a group where values are alike. Beverages sell much more than Merchandise, so the outlet average mixes very different items and makes normal Beverages days look huge. Grouping by Item avoids this.

7. The two-chart dashboard

There is no printed result. Open outlets_and_beverages.html. Pune should have the tallest bar, and the Beverages line should dip in February and rise to a peak in December. Check hover and zoom.

8. The Streamlit app

Four small changes to app.py. The new lines are marked with a comment.

```
# In section 5 (filters), after the month slider:
low_offer, high_offer = st.sidebar.slider("Offer (%)", 0, 25, (0, 25))   # new

# In section 6 (matching rows), add a third condition:
             & clean["Offer"].between(low_offer, high_offer)]             # new

# In section 8 (headline numbers), use four columns and add a fourth number:
col1, col2, col3, col4 = st.columns(4)                                    # changed
col4.metric("Best outlet", view.groupby("Outlet")["Revenue"].sum().idxmax())   # new
```

To check it: with all filters on, the app shows 3,282 rows, 920.6 million and Pune as the best outlet. Move the offer slider to 20 to 25 and it shows 1,079 rows, 326.4 million and Pune. Then remove Pune and the best outlet becomes Kochi (712 rows, 205.0 million). Each condition starts with &, which means “and”. Only the last condition ends with the closing bracket], so move that bracket to the end of the new line.

9. A sample report

A sample is the “Example report” in section 9. Check that yours has all four parts, a number in each finding, and recommendations that start with a verb. Different findings are fine, as long as every number can be found in one of your charts.